MindMemory

Memory for Claude Code that gets better the more you use it.

PANEL A · CONCEPT

From agents to minds.

Capability has plateaued; **memory is the next lever**. A "mind" is identity + memory + heuristics, wired together as one markdown construct.

01

Agents

`stateless`

- High capability per call
- No memory
- No personality
- Transactional

02

Claude Code today

`session-scoped`

- High capability
- Session-only memory
- Thin prompted personality
- Transactional

MINDMEMORY

03

A mind

`colleague-like`

- High capability
- Persistent memory
- Stable identity
- Growing heuristics

04

Human mind

`aspiration`

- Variable capability
- Lifelong, selective
- Deep, stable
- Continuous learning

A mind is one markdown-based construct: `identity` + `memory` + `heuristics`. **Claude Code is the substrate. Markdown is the storage.** No vector DB, no fine-tuning.

PANEL B · MEMORY

Four tiers, by lifespan.

Organised by how long things live, not by topic. Index pattern keeps always-loaded context to ~1–2 KB while making thousands of memories available on demand.

B1 · The 4-tier stack

TOP = LONGER-LIVED

01 **Identity**
Personality. Rules it never breaks.
`identity/anchor.md`
READ-ONLY

02 **Long-term memory**
Confirmed patterns, profile, lessons.
`memory/MEMORY.md · {topic}.md`
WEEKS+

03 **Working memory**
This week — focus, blockers, recent corrections.
`memory/working-memory.md`
DAYS

04 **Archive**
Compressed history. Not loaded by default.
`memory/archive.md`
RARELY READ

B2 · Hunch → heuristic

3-CONFIRMATION RULE

1× **Observation** `logged`
Noted by collector. Not yet in memory.

2× **Hunch** `tentative`
Added to working-memory, labelled.

3× **Heuristic** `permanent`
Promoted to Tier 2. Applied going forward.

↺ **Contradicted** `demoted`
Returns to hunch, or archived with note.

Each Tier-2 entry stores **why** + **how to apply** — so the mind can judge edge cases instead of blindly following.

PANEL C · SELF-MAINTENANCE

Three concentric loops.

The mind curates itself **while you work**, not just at logoff. Catching corrections within minutes produces tighter rules than catching them hours later.

INNER LOOP

Mid-session — every N tool calls

PostToolUse hook

Collect signals

`memory-collector.py`

Every **~10 calls**: scans live transcript, appends to `working-memory.md` while context is hot.

- correction
- confirmation
- follow-through
- frustration

→

Promote / demote

`memory-curator.py`

Every **~25 calls**: applies the 3-confirmation rule. Hunch → heuristic. Contradicted → demoted. Stale (>5d) → archived.

MIDDLE LOOP

Per session — bookends

SessionStart · SessionEnd

Session start

`session-start-health.py`

Loads **identity** + **MEMORY.md** + **working-memory**. Warns if curator hasn't run in >7 days.

→

Session end

`session-metrics.py`

Writes one row to `session-metrics.csv`. Heavy maintenance already happened in the inner loop — this is the safety net.

OUTER LOOP

Weekly — time-gated

7-day timestamp

Compute hypotheses

`metrics-brain.py`

Reads CSV, computes **H1–H5**, writes `daily-report.md`. Measurement keeps the system honest.

→

Regenerate index `memory-index-weekly.py`

Optional. Rebuilds `memory/INDEX.md` grouped by rules + decisions.

Why nest them? **By the time a session ends, the model has drifted from the moment of insight.** The inner loop catches corrections within minutes; bookends and weekly review are safety nets and measurement.

PANEL D · FINDINGS

Does it actually help?

9,869 sessions over 3 months. Matched cohort: **71 with memory** vs **21 without** (≥30 tool calls, post-instrumentation).

H1

Production ratio

18.5% vs 8.5%

WIN — 2.2×

H2

Calls before first edit

14.6 vs 16.3

WIN — FASTER ORIENT

H4

Productive calls / session

26.0 vs 6.0

WIN — 4.3×

H5

Repeat-correction rate

19.7% vs 14.3%

NULL — PRIMARY OUTCOME

Memory-loaded sessions are denser and orient faster. But H5 — the headline outcome (fewer repeated corrections) — came back null. We don't have a clean A/B yet. Publishing specifically to get more data and pushback on experimental design.